01

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

요청 하나의 일생

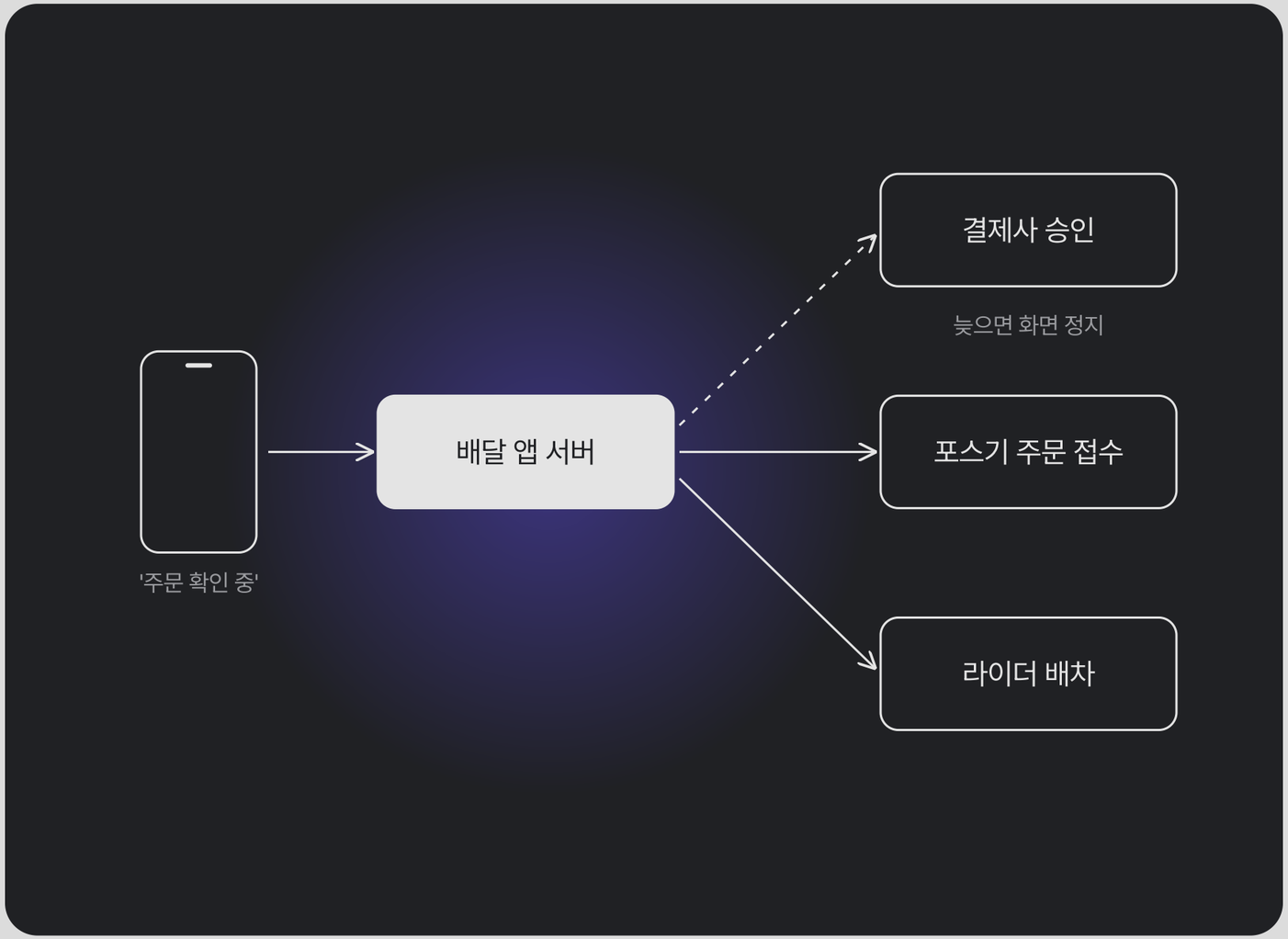
제1부 기반: 백엔드와 Node.js 런타임 · 1장 오리엔테이션

버튼 한 번에 떠난 요청, 어디를 거치고 어디서 멈추나



© 한상훈 NestJS 백엔드 마스터

'주문 확인 중'에서 멈춘 화면



→ 주문하기 → 배달 앱 서버

→ 배달 앱 서버 → 결제사 승인, 가게 포스기 주문 접수, 라이더 배차

→ 셋 중 하나만 늦어도 화면 정지

→ 손님이 보는 것: 화면 하나

© 한상훈 NestJS 백엔드 마스터

화면에 띄울 말은 서버가 정함

1

계속 대기

2

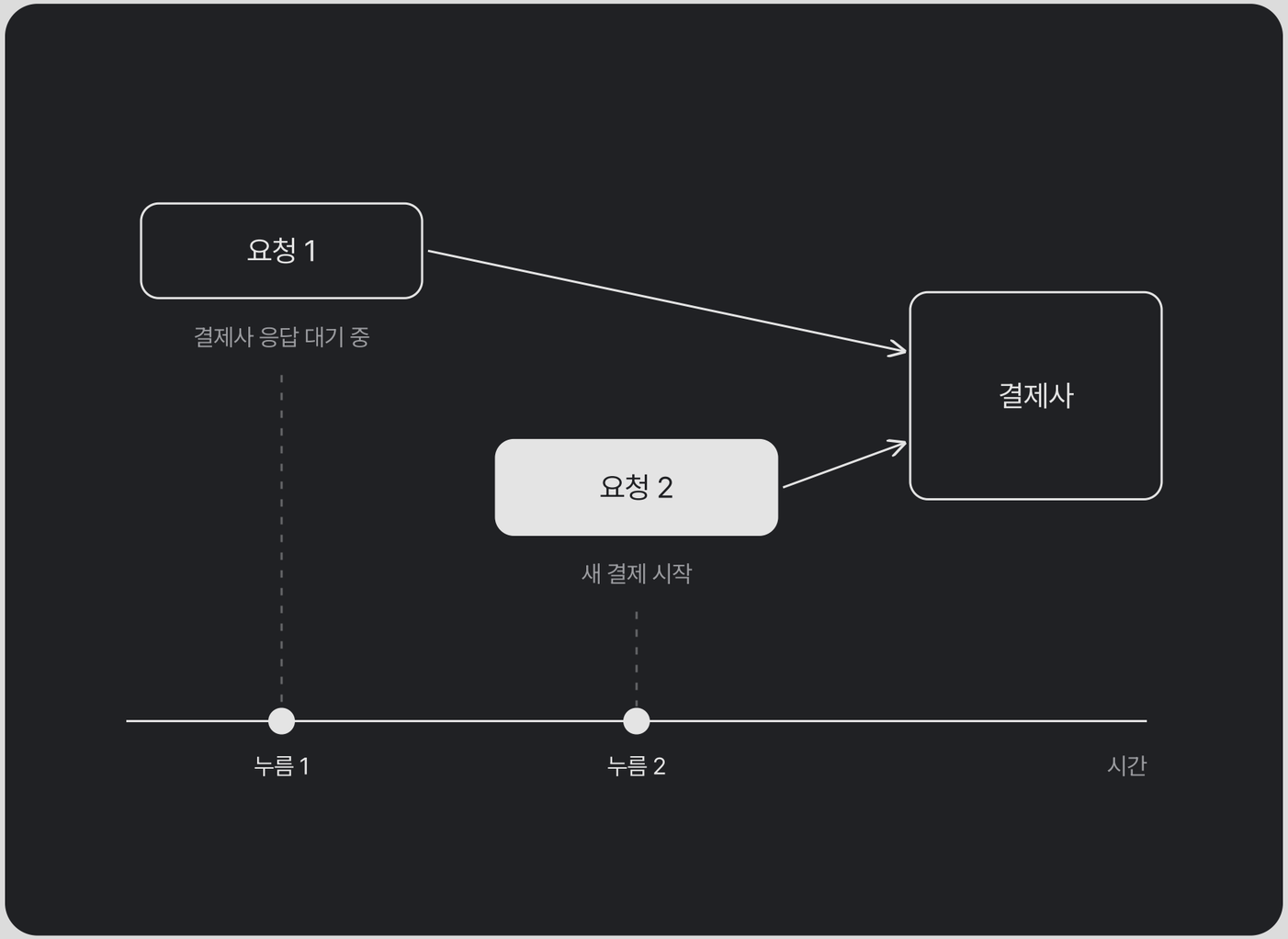
'잠시 후 다시 시도해 주세요'

3

결제 먼저 확정, 나머지는 나중에 알림

© 한상훈 NestJS 백엔드 마스터

주문하기를 한 번 더 누르면



→ 요청 1: 결제사 응답 대기 중

→ 요청 2: 새 결제 시작

→ 결과: 저녁 한 끼에 결제 두 번 가능

© 한상훈 NestJS 백엔드 마스터

고객센터 단골 문의 두 개

문의

"결제는 됐는데 주문이 안 들어갔어요"

문의

"한 번 눌렀는데 두 번 결제됐어요"

→ 로그 속 원인: 몇 초 늦은 결제사 응답, 다시 누른 버튼

→ 코드 한 줄씩은 정상 동작, 결과는 오류

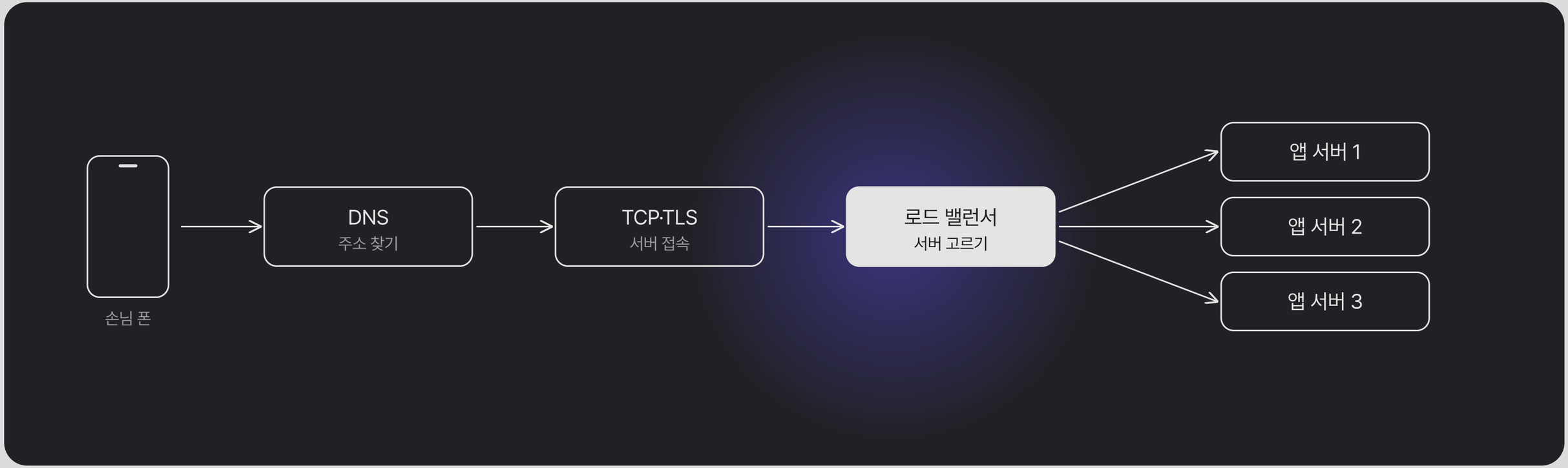
© 한상훈 NestJS 백엔드 마스터

서버 코드 앞의 세 구간

→ 주소 찾기: DNS

→ 서버 접속: TCP 접속(패킷 세 번 왕복), TLS 핸드셰이크

→ 서버 고르기: 로드 밸런서



© 한상훈 NestJS 백엔드 마스터

DNS: 이름을 IP 주소로

```
api.delivery.com → 203.0.113.10
```

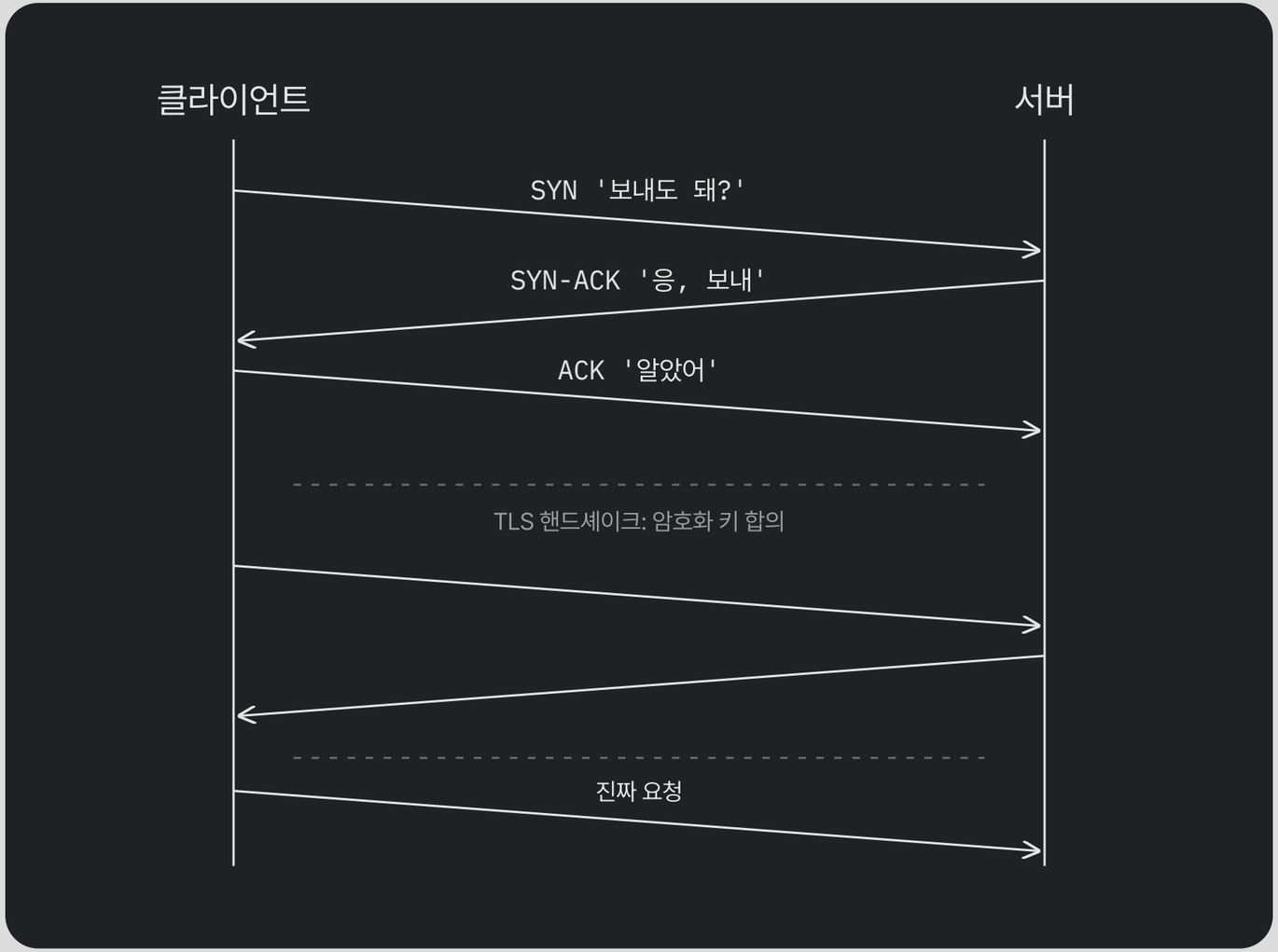
→ 203.0.113.10: 예시 주소

→ DNS(Domain Name System): 이름을 IP 주소로 바꾸는 시스템

→ 전화번호부에서 이름으로 번호 찾기와 같은 일

© 한상훈 NestJS 백엔드 마스터

TCP 접속과 TLS 핸드셰이크



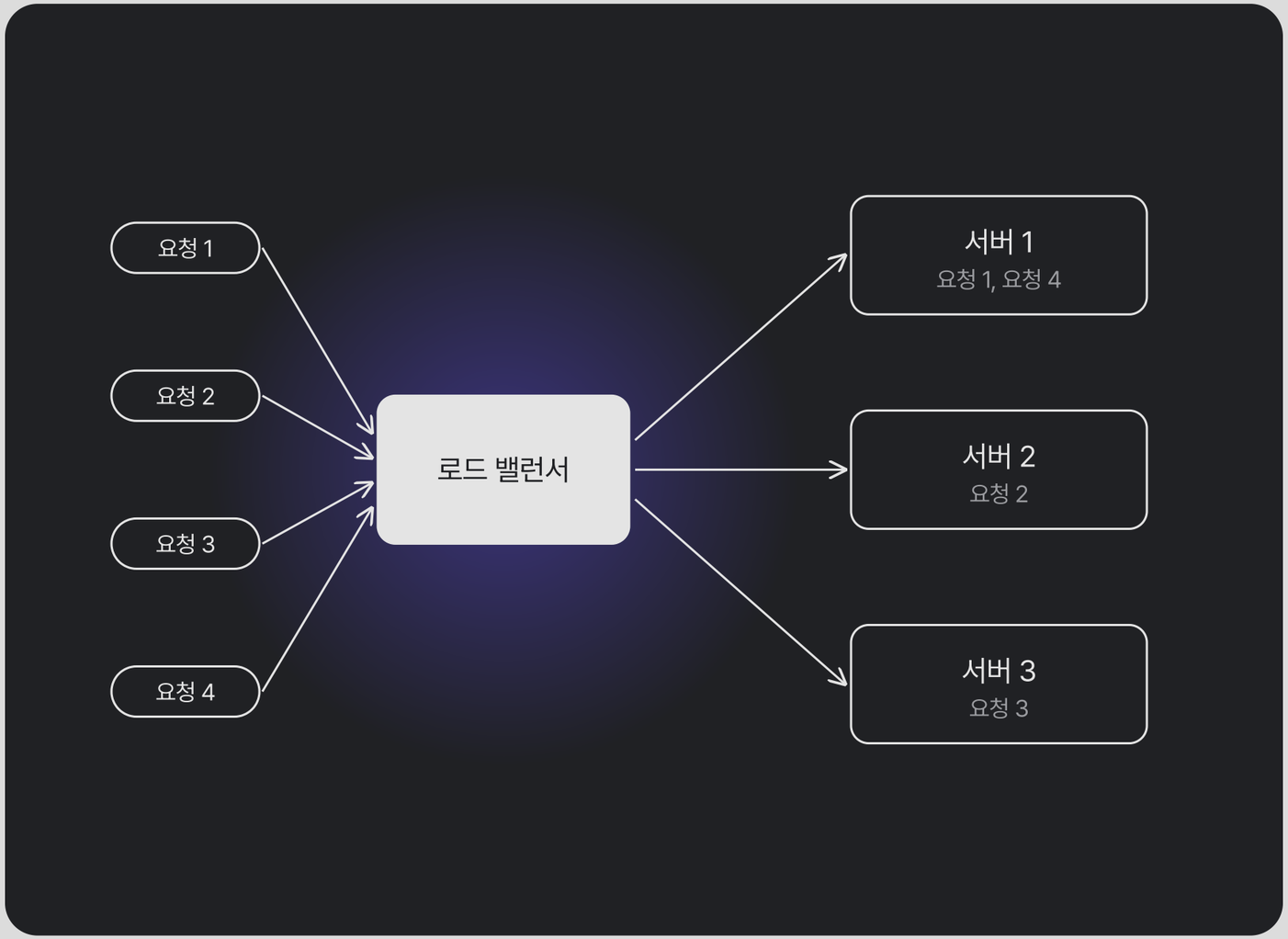
→ TCP 접속: '보내도 돼?' → '응, 보내' → '알았어', 패킷 세 번 (SYN, SYN-ACK, ACK)

→ TLS 핸드셰이크: 암호화 키 합의, 왕복 추가

→ https 요청 = TCP 접속 뒤 TLS

© 한상훈 NestJS 백엔드 마스터

로드 밸런서



→ 같은 앱 서버 여러 대 앞의 분배기

→ 요청 1 → 서버 1, 요청 2 → 서버 2, 요청 3 → 서버 3, 요청 4 → 서버 1

→ 같은 순간의 두 주문, 다른 서버로 갈 수 있음

→ 같은 손님의 두 번 누른 버튼도 마찬가지

© 한상훈 NestJS 백엔드 마스터

curl -w로 구간 재기

터미널

```
$ curl -s -o /dev/null -w "dns=%{time_namelookup} connect=%{time_connect} tls=%{time_appconnect}
ttfb=%{time_starttransfer} total=%{time_total} code=%{http_code}\n" https://example.com/
```

→ 각 값: 요청 시작부터 그 구간이 끝난 시각까지의 누적 초

→ ttfb: 첫 바이트 도착 시각

© 한상훈 NestJS 백엔드 마스터

example.com 요청 한 번

구간	끝난 시각	구간 길이
이름 찾기	0.248초	0.248초
연결	0.498초	0.250초
TLS	0.743초	0.245초
첫 바이트	0.977초	0.234초

→ 서버 코드가 도는 곳: 마지막 0.234초 안

→ 앞의 4분의 3: 서버 코드 실행 전에 쓴 시간

→ 지하철 LTE: 패킷 왕복 시간 증가, 연결·TLS 구간이 거의 비례해서 증가

데이터 블록: 2026-10-25 집 와이파이에서 실측, curl 8.16.0, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

example.com과 localhost

구간	example.com	localhost
이름 찾기	0.248초	0.0001초
연결	0.498초	0.003초
TLS	0.743초	없음
전체	0.977초	0.012초

→ 로컬 개발: 세 구간이 거의 0

데이터 블록: 2026-10-25 실측, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

같은 요청을 한 번 더

구간	첫 요청	바로 다음 요청
이름 찾기	0.248초	0.018초
연결	0.498초	0.246초
TLS	0.743초	0.486초

→ 이름 찾기: 찾은 주소를 잠깐 기억해서 짧아짐

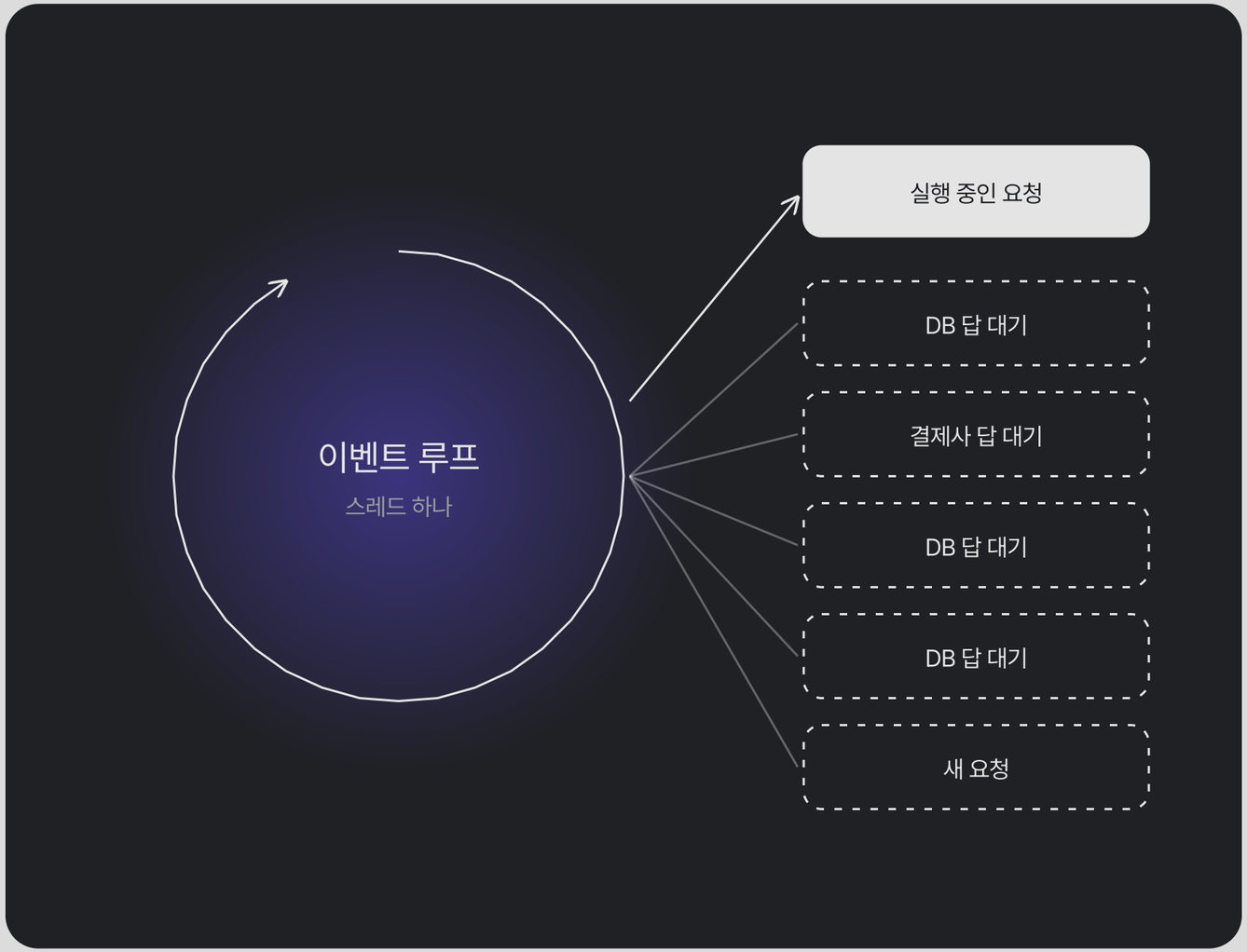
→ 연결과 TLS: curl이 요청마다 연결을 끊어서 매번 새로

→ 연결을 열어 두고 다시 쓰면 두 구간 절약

데이터 블록: 2026-10-25 실측, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

Node 스레드 하나와 이벤트 루프



→ 자바스크립트 실행 스레드: 하나

→ 이벤트 루프: 새 요청 처리, 도착한 응답의 다음 코드 실행을 반복하는 반복문

→ 요청 백 개 중 대부분: DB나 다른 서버의 답 대기

→ 대기 중에는 스레드 미사용, 그사이 다른 요청 처리

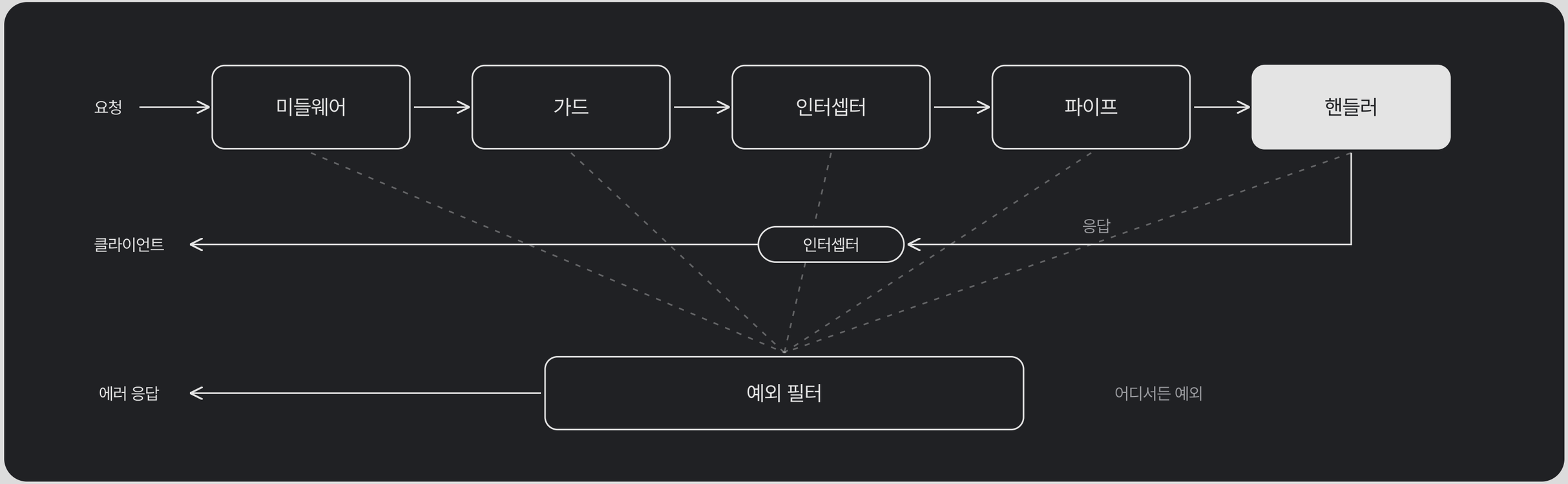
© 한상훈 NestJS 백엔드 마스터

NestJS 요청 관문의 순서

→ 미들웨어 → 가드 → 인터셉터 → 파이프 → 핸들러(컨트롤러 메서드)

→ 응답: 핸들러 → 인터셉터 → 클라이언트

→ 어디서든 예외 → 예외 필터 → 에러 응답



© 한상훈 NestJS 백엔드 마스터

주문 API의 관문

관문	하는 일
미들웨어	들어온 시각과 경로 로그
가드	로그인한 사용자인지 확인
파이프	메뉴 ID가 숫자인지, 수량이 0보다 큰지 검사
핸들러	주문 처리

→ 컨트롤러: 로그인·수량 재확인 불필요

© 한상훈 NestJS 백엔드 마스터

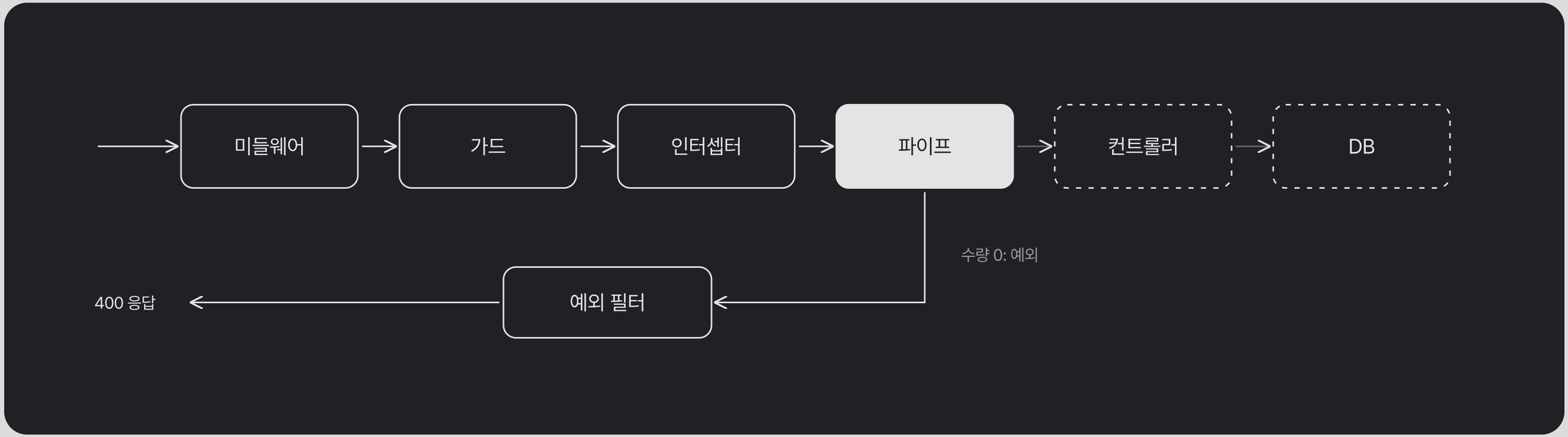
수량 0인 주문 요청

→ 파이프에서 예외

→ 예외 필터 → 400 응답

→ 컨트롤러, DB 미도달

→ 잘못된 요청을 앞에서 돌려보낸 만큼 DB 부담 감소



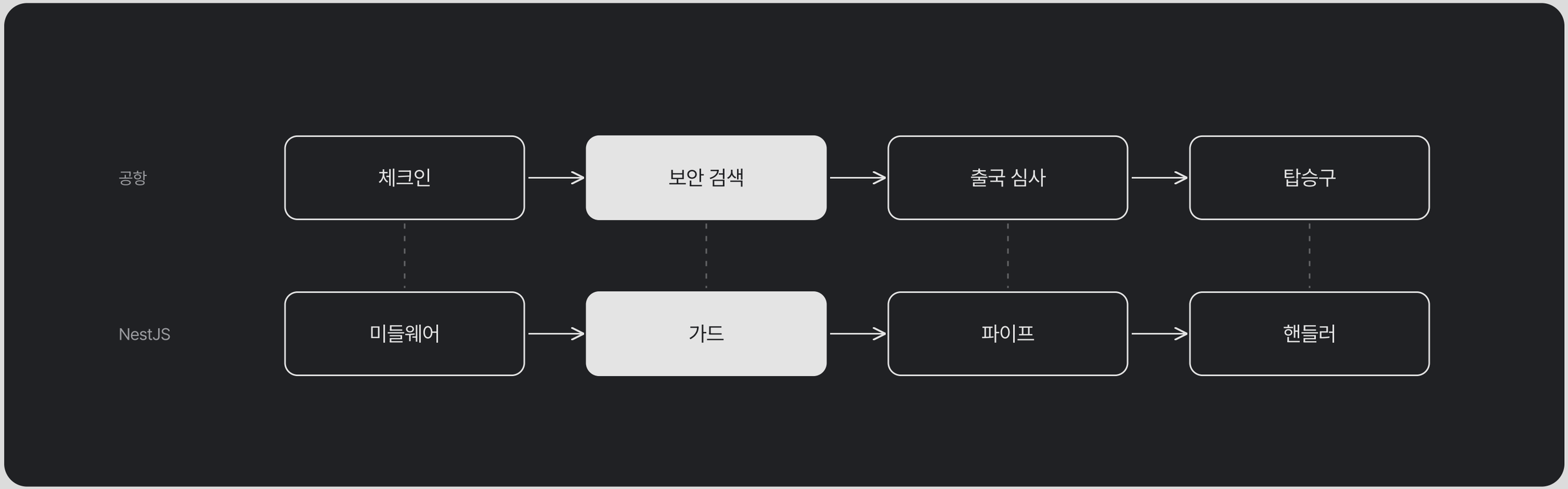
© 한상훈 NestJS 백엔드 마스터

공항 출국 순서

→ 체크인 → 보안 검색 → 출국 심사 → 탑승구

→ 보안 검색 없이 면세점 입장 불가

→ NestJS: 가드 불통과 요청은 파이프까지 가지 않음



© 한상훈 NestJS 백엔드 마스터

관문이 DB를 부르면

관문 하나

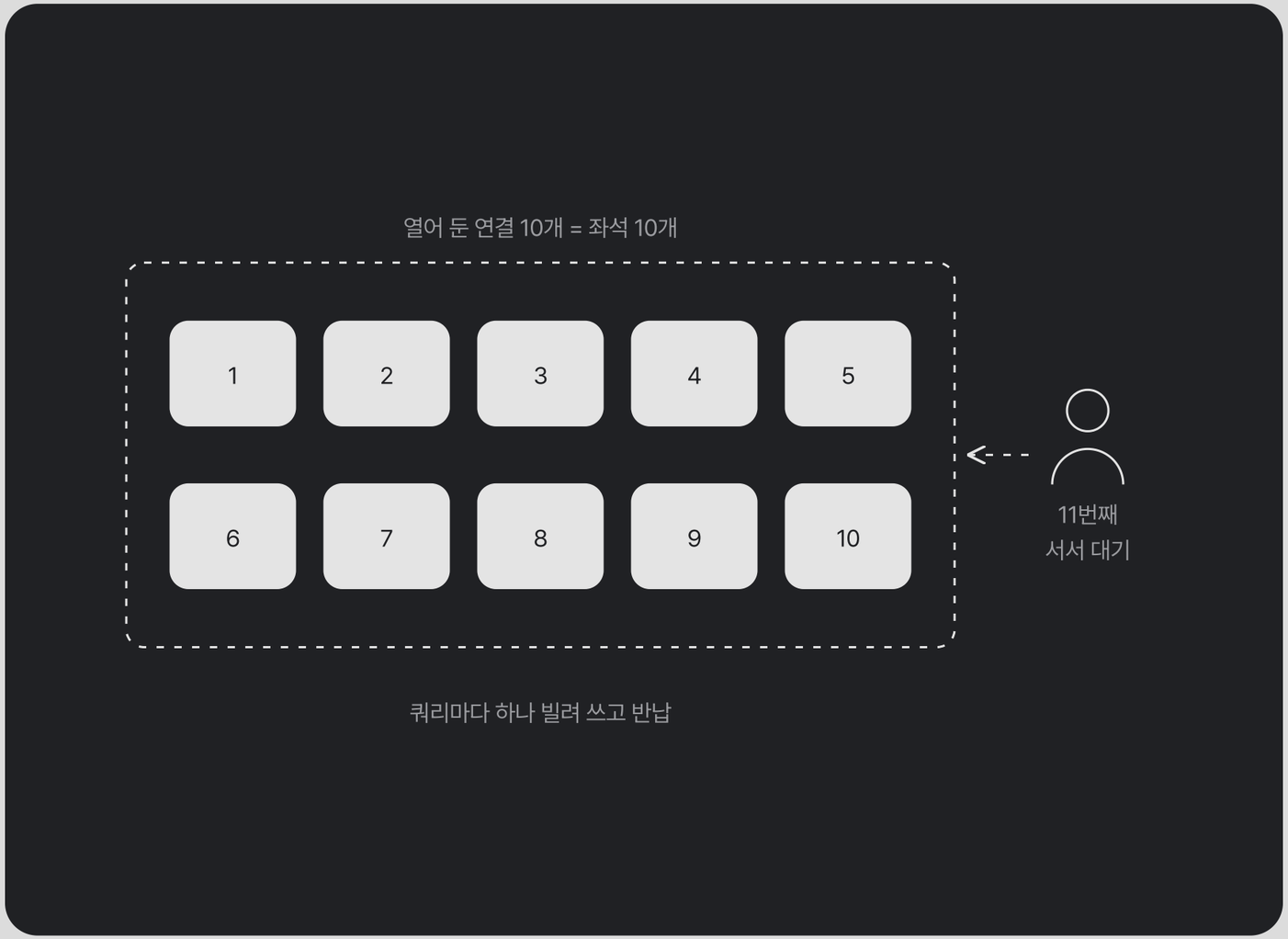
대부분 1밀리초 미만

가드에서 사용자 정보 DB 조회

관문 하나가 DB 왕복 한 번

© 한상훈 NestJS 백엔드 마스터

커넥션 풀



→ DB 연결 비용: 새 접속, 로그인, 권한 확인

→ 앱 시작 때 연결 여러 개를 미리 열어 둠

→ 쿼리마다 하나 빌려 쓰고 반납

→ 커넥션 풀(connection pool): 열어 둔 연결 묶음

© 한상훈 NestJS 백엔드 마스터

풀 10개, 쿼리 20밀리초

항목	값
연결 하나의 초당 쿼리	1,000밀리초 ÷ 20밀리초 = 50번
연결 10개의 초당 쿼리	500번
요청당 쿼리 1번일 때 초당 요청	500개

데이터 블록: 풀 크기와 쿼리 시간은 계산용 가정값

© 한상훈 NestJS 백엔드 마스터

가드에 쿼리 하나 추가

요청당 쿼리	초당 받을 수 있는 요청
1번	500개
2번 (가드의 사용자 조회 추가)	250개

→ 코드 한 줄로 처리량 절반

© 한상훈 NestJS 백엔드 마스터

1초에 1,000개가 오면

지난 시간	줄 선 요청	맨 뒤 요청의 대기
1초	500개	1초
5초	2,500개	5초
10초	5,000개	10초

→ 바로 에러 대신 대부분 대기

→ 쿼리는 그대로 20밀리초, 응답은 10초

© 한상훈 NestJS 백엔드 마스터

한가한 CPU, 멈춘 화면

줄 선 요청

계산 없이 연결만 대기

CPU 사용률 그래프

한가해 보임

손님 화면

'주문 확인 중'

→ 느려지는 이유: CPU보다 연결 열 개인 경우가 많음

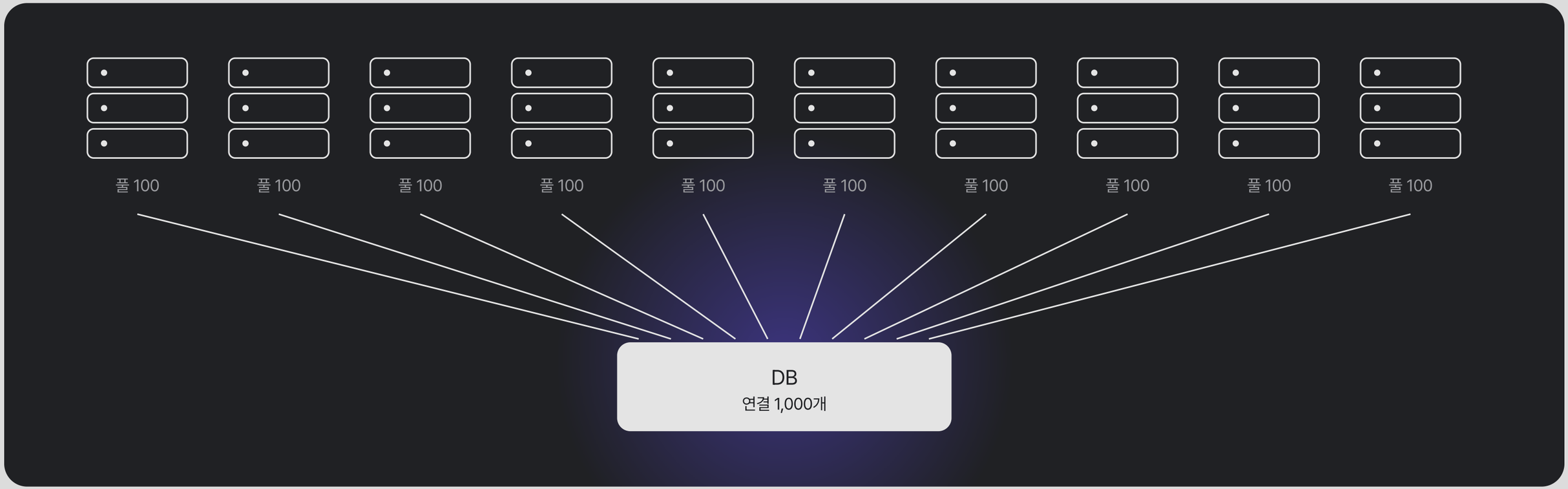
© 한상훈 NestJS 백엔드 마스터

서버 열 대의 커넥션 풀

→ 서버 10대 × 풀 100개 = DB 연결 1,000개

→ DB: 연결마다 메모리 사용, 받을 수 있는 연결 수에 한도

→ 풀 크기 = DB가 받아 주는 연결 수 ÷ 서버 대수



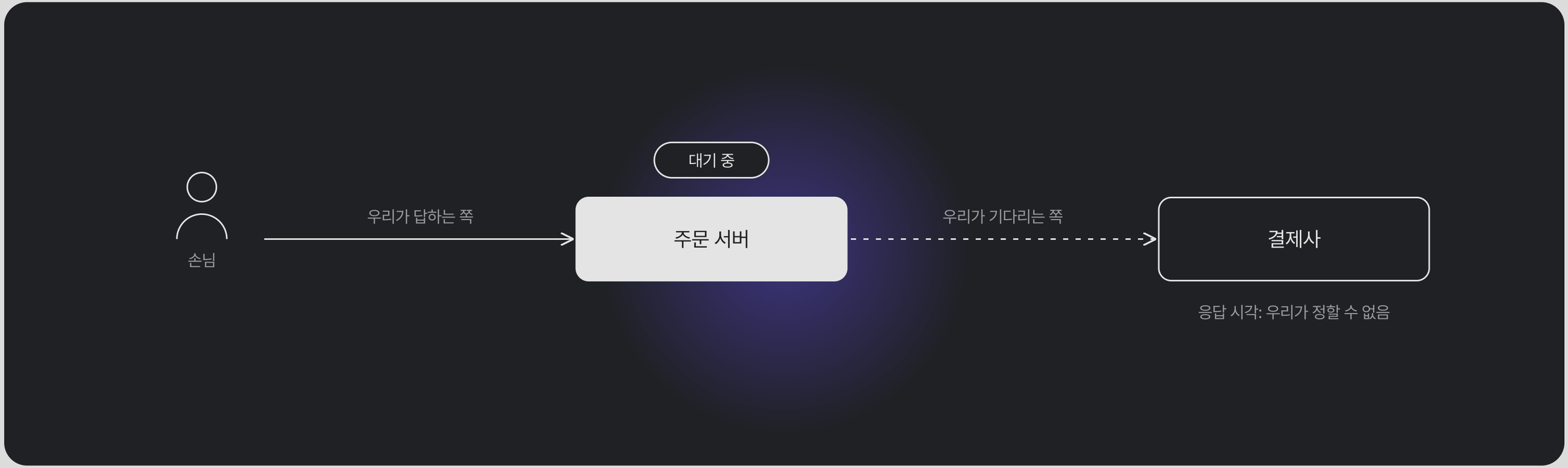
© 한상훈 NestJS 백엔드 마스터

서버가 손님이 되는 순간

→ 손님 → 우리 서버: 우리가 답하는 쪽

→ 우리 서버 → 결제사: 우리가 기다리는 쪽

→ 결제사 서버의 응답 시각: 우리가 정할 수 없음



© 한상훈 NestJS 백엔드 마스터

답하지 않는 가짜 결제사

```
// 요청은 받지만 끝내 답하지 않는 가짜 결제사
const http = require('node:http');
http.createServer((req, res) => {
  console.log(new Date().toISOString(), req.method, req.url, '받음, 답하지 않음');
}).listen(4000, () => console.log('fake payment on 4000'));
```

© 한상훈 NestJS 백엔드 마스터

결제사를 부르는 주문 서버

```
// 주문 API: 결제사 호출 뒤 응답 대기
const http = require('node:http');
let pending = 0;
http.createServer(async (req, res) => {
  pending++;
  const started = Date.now();
  console.log(`주문 받음, 결제 대기 중인 요청 ${pending}개`);
  const r = await fetch('http://localhost:4000/approve', { method: 'POST' });
  pending--;
  res.end(`결제 응답 ${r.status}, ${Date.now() - started}ms\n`);
}).listen(3001, () => console.log('order api on 3001'));
setInterval(() => console.log(`[${new Date().toISOString().slice(11,19)}] 결제 대기 중 ${pending}개`), 5000).unref();
```

© 한상훈 NestJS 백엔드 마스터

5초 뒤 떠난 curl, 남은 요청

○ ○ ○ 터미널

```
$ curl -s -m 5 -o /dev/null -w "code=%{http_code} total=%{time_total}\n" -X POST http://localhost:3001/order
code=000 total=5.007149
(curl exit 28: 시간 초과)
```

○ ○ ○ 주문 서버 로그

```
order api on 3001
주문 받음, 결제 대기 중인 요청 1개
[03:21:37] 결제 대기 중 1개
[03:21:42] 결제 대기 중 1개
[03:21:47] 결제 대기 중 1개
```

→ 손님은 떠났고, 서버는 결제사 연결 하나와 메모리를 계속 차지

© 한상훈 NestJS 백엔드 마스터

fetch가 기다린 시간

터미널

```
실패 303초 뒤: UND_ERR_HEADERS_TIMEOUT
```

→ 응답 헤더가 오지 않아 포기

→ 기본 대기: 응답 헤더 5분

→ 따로 정하지 않으면 5분

데이터 블록: Node v24.11.0 내장 fetch 실측, 기본값은 Node 문서 기준, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

결제사가 멈춘 동안 쌓이는 요청

지난 시간	기다리는 요청 (초당 주문 500개)
1초	500개
1분	3만 개
5분	15만 개

→ 요청마다 연결 하나와 메모리

→ 메모리 바닥 → 결제와 무관한 메뉴 조회 API까지 중단

→ 다시 누른 주문도 다시 5분 대기

© 한상훈 NestJS 백엔드 마스터

정해야 할 두 가지

1

몇 초까지 기다릴지

2

그 시간이 지나면 손님에게 무슨 응답을 줄지

→ 응답 없음 ≠ 결제 안 됨

→ 결제사 승인 완료 뒤 돌아오는 길에 사라진 응답 가능

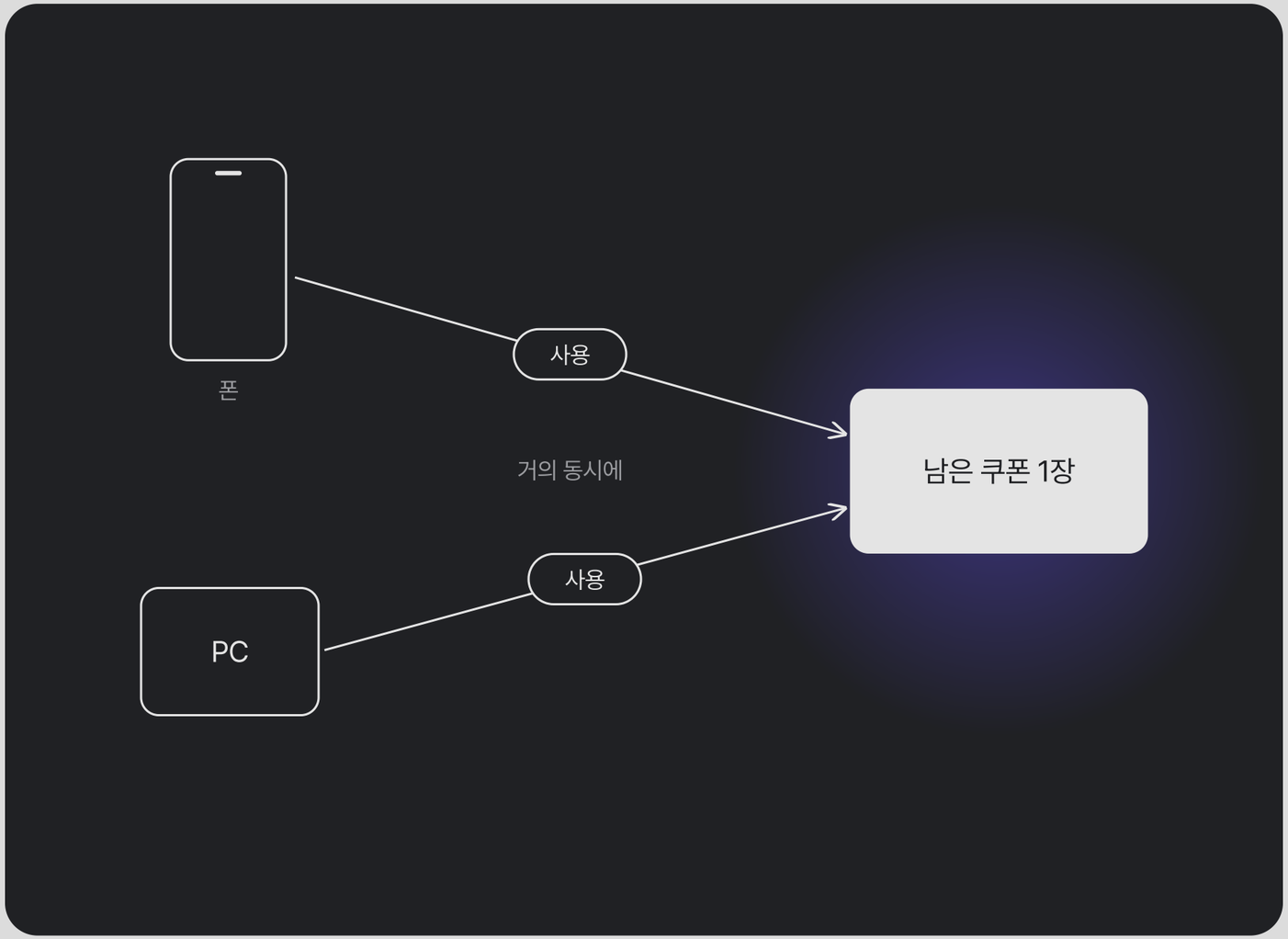
© 한상훈 NestJS 백엔드 마스터

로컬에서 맞고 운영에서 틀리는 감각

감각	로컬	운영
요청은 하나씩	Postman으로 한 번씩	1초에 수천 개가 풀 10개를 나눠 씀
네트워크는 늘 응답	localhost, 연결 0.003초	결제사 무응답, 지연, 응답 유실
DB는 하나	도커 DB 하나	쓰기 한 대, 읽기 복사본 여러 대

© 한상훈 NestJS 백엔드 마스터

동시에 누른 쿠폰



→ 남은 쿠폰: 1장

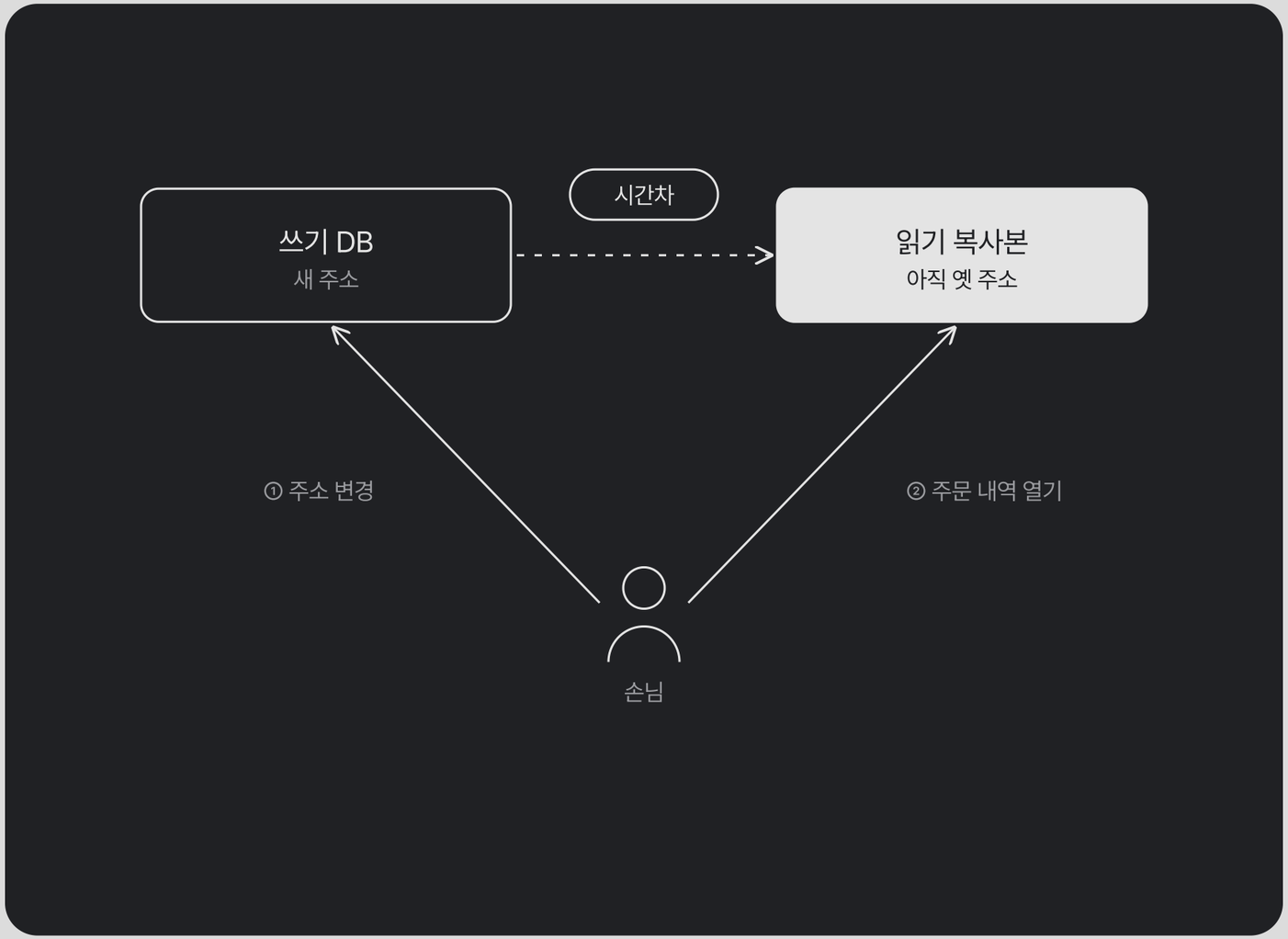
→ 폰과 PC에서 거의 동시에 '사용'

→ 두 요청이 같은 쿠폰을 동시에 사용 시도

→ 로컬에서는 일어나지 않는 일

© 한상훈 NestJS 백엔드 마스터

방금 바꾼 주소



→ 주소 변경 → 쓰기 DB

→ 바로 주문 내역 열기 → 읽기 복사본

→ 복사본에 아직 안 넘어온 새 주소 → 옛 주소 표시

→ 평소 차이는 거의 0, DB가 바쁜 날 눈에 보임

© 한상훈 NestJS 백엔드 마스터

로컬 테스트가 통과하는 이유

테스트

요청을 하나씩

가짜 결제사

늘 바로 응답

DB

하나

→ 운영에서 처음 터지고, 로컬에서는 재현이 어려움

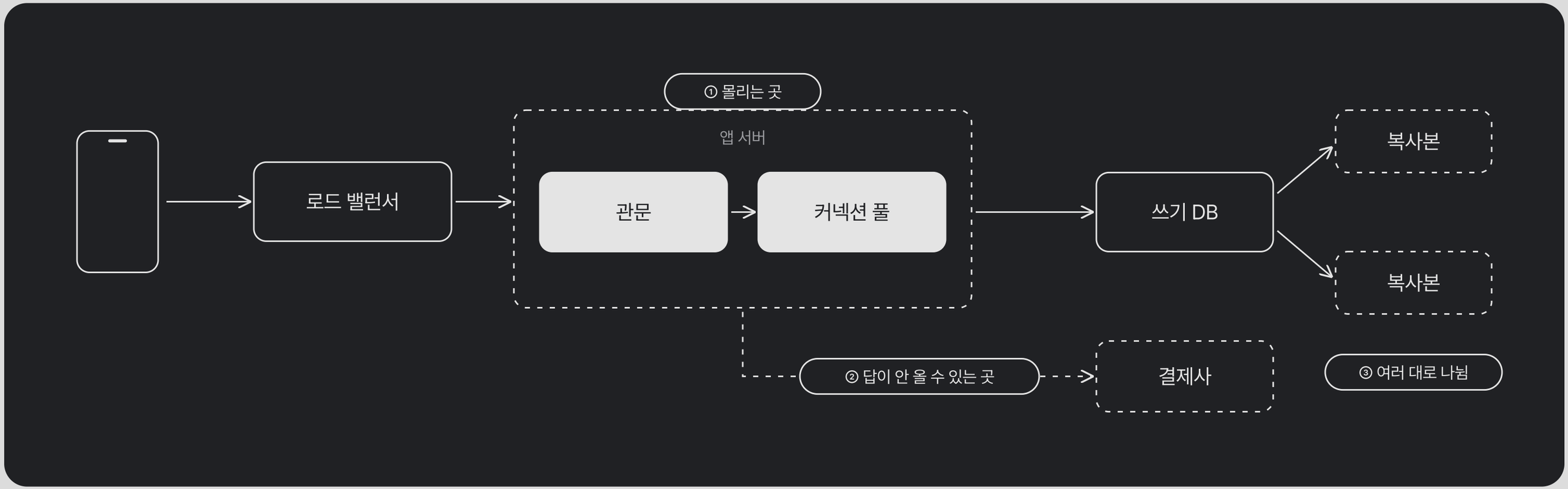
© 한상훈 NestJS 백엔드 마스터

요청의 일생 위 세 표시

→ 관문과 커넥션 풀: 요청이 몰리는 곳

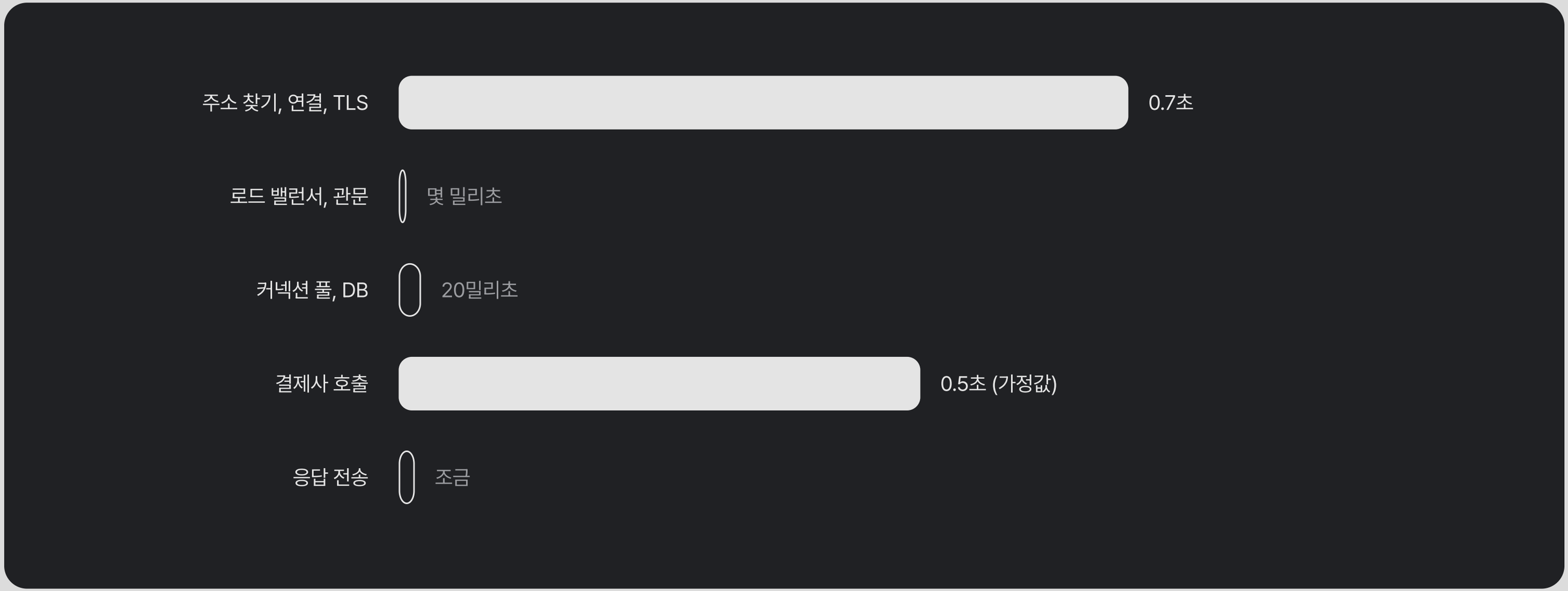
→ 결제사로 나가는 화살표: 답이 안 올 수 있는 곳

→ DB: 여러 대로 나뉘는 곳



© 한상훈 NestJS 백엔드 마스터

구간별 시간



→ 합계: 1초 남짓

© 한상훈 NestJS 백엔드 마스터

1밀리초와 0.2초

고친 곳	줄어든 시간	1초 중 비율
관문 코드 2밀리초 → 1밀리초	1밀리초	0.1%
결제사 호출 0.5초 → 0.3초	0.2초	20%

© 한상훈 NestJS 백엔드 마스터

결제사가 멈춘 날의 막대

결제사 막대

0.5초 → 5분

배달 앱

포스기 0.1초, 라이더 배차 3분 → 화면은 3분 내내 '주문 확인 중'

→ 나머지 구간 속도와 무관하게 5분 대기 또는 이탈

© 한상훈 NestJS 백엔드 마스터

구간마다 시각 찍기

바깥 구간

`curl -w`

안쪽 구간

결제사 호출 직전·직후, 쿼리 직전·직후 시각을 로그로

→ 구간 다섯 개 → 숫자 다섯 개, 가장 큰 숫자가 고칠 곳

→ 쌓인 숫자: 평균 말고 가장 느린 요청들만 따로 보기

© 한상훈 NestJS 백엔드 마스터

평소 응답 시간과 최악의 응답 시간

구분	정하는 구간
평소 응답 시간	가장 느린 구간
최악의 응답 시간	기다리는 시간을 정하지 않은 구간

© 한상훈 NestJS 백엔드 마스터

요청 하나의 일생

"요청 하나는 여러 곳을 거치고,
그중 가장 느린 곳이 응답 시간을 정합니다."